Compiladores e Intérpretes
Analizador Semántico – Árbol Sintáctico Abstracto (AST) – Diseño y Construcción

Universidad Nacional del Sur
Departamento de Ciencias e Ingeniería de la Computación
2026

Análisis Semántico - Intuición

- El analizador **semántico** tomará las **estructuras generadas** por el sintáctico y
  - Terminará de **recopilar la información** del programa y las **relaciones** entre las **entidades**
  - **Controlar** que el programa es **semanticamente valido**
- Es usual agrupar estas tareas según el nivel donde se realizan...

Chequeo de Declaraciones

Chequeo de Sentencias

Análisis Semántico - Intuición

- El analizador **semántico** tomará las **estructuras generadas** por el sintáctico y
  - Terminará de **recopilar la información** del programa y las **relaciones** entre las **entidades**
  - **Controlar** que el programa es **semanticamente valido**
- Es usual agrupar estas tareas segú... ...

Chequeo de Declaraciones

En el chequeo de declaraciones la estructura que utilizamos es la **Tabla de Símbolos**. Esta estructura es una representación alternativa de las entidades declaradas en el programa fuente

Estructura para el Chequeo de Sentencias

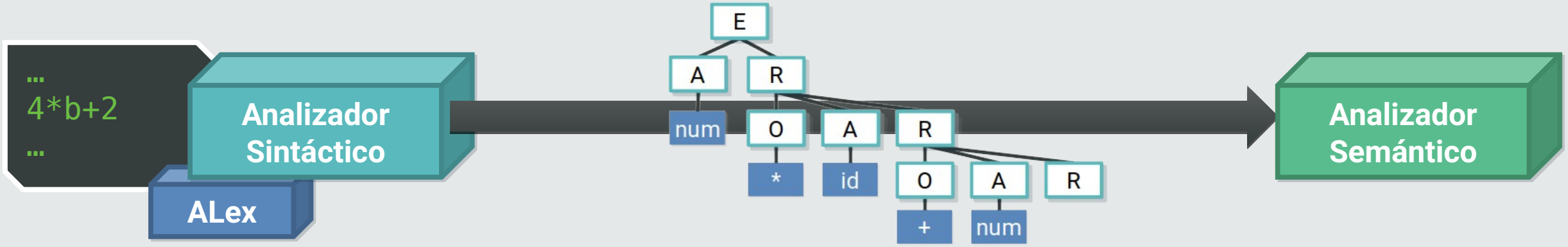
- El analizador **semántico** tomará las **estructuras generadas** por el sintáctico y
  - Terminará de **recopilar la información** del programa y las **relaciones** entre las **entidades**
  - a es **semanticamente valido**
  - s según el nivel donde se realizan...

En el chequeo de sentencias debemos controlar las sentencias del programa (lo que aparece dentro de los bloques)... Por lo tanto necesitamos una estructura que nos provea una **representación alternativa de las sentencias**

Chequeo de Sentencias

Estructura para el Chequeo de Sentencias

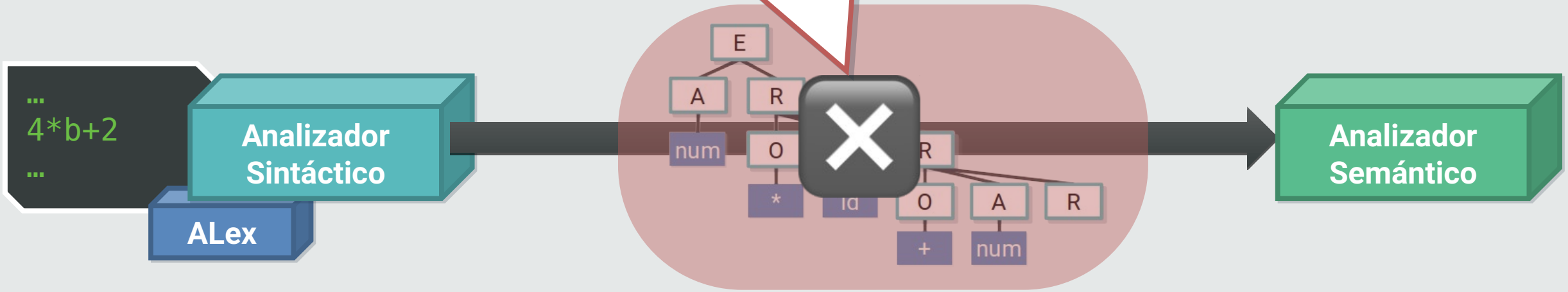
- Para el que **chequeo de sentencias…** ¿Qué **representación alternativa** del fuente usamos?
  - Una **opción** sería que el sintáctico construya el **árbol de derivación completo** para el código de cada función/método
    - Lo tenemos “automáticamente” ya que el analizador sintáctico lo simula!



Estructura para el Chequeo de Sentencias

El **problema** es su **tamaño** y que tiene **muchísimas cosas** que **no** son necesarias para entender el **significado** (ie realizar el análisis semántico).

- P… …ué **representación** **a**…
  - …a el **árbol de derivación** **completo** para el código d… …nción/método
    - Lo tenemos “automáticamente” … …e el analizador sintáctico lo simula!



Estructura para el Chequeo de Sentencias

- El **árbol de derivación** (o la misma **gramática** que utilizamos para el **análisis sintáctico**) tiene **muchos componentes** que **no contribuyen al significado** del programa
- Por ejemplo:
  - **NT** para indicar que un **bloque** tiene una **lista de sentencias**
  - Todos los **NT** usados cuando algo puede ser vacío o no
  - Los **NT** que introducimos para **eliminar la recursión a izquierda**
  - Los **delimitadores**!

Arboles Sintácticos Abstractos (ASTs)

Arboles Sintácticos Abstractos

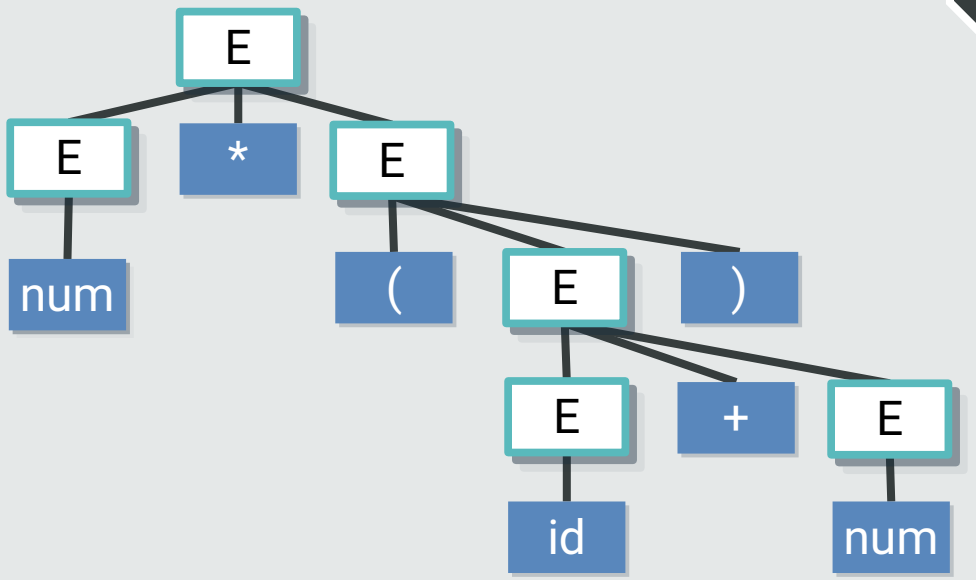
- Los **árboles sintácticos abstractos** (ASTs) son una forma arbórea de **representar** la **estructura sintáctica concreta** de un programa.
- A **diferencia** de los **arboles de derivación** en el **AST** cada nodo es un componente/constructor sintáctico que **efectivamente aparece** y tiene un **significado** en el **código** del programa
- La palabra **abstracto** esta asociada a que en estos arboles nos abstraemos de algunos componentes sintácticos que no son útiles para el semántico o que quedan implícitamente modelados:
  - Por ejemplo: paréntesis, puntos y comas, comas, etc.

Arboles Sintácticos Abstractos

- Diferencias entre el árbol de derivación y el AST

E → E+E | E*E |
(E) | id | num

```
...
4*(b+2)
...
```

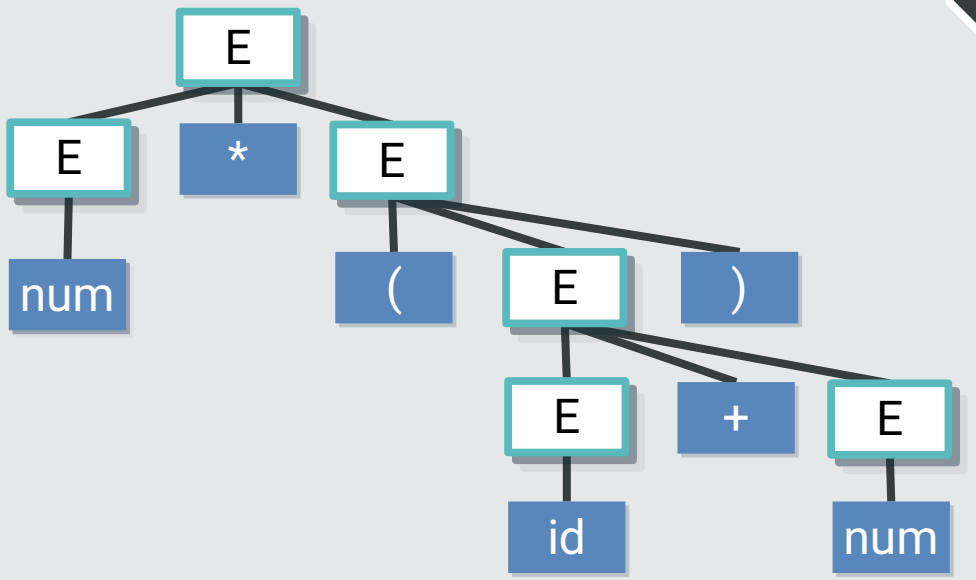


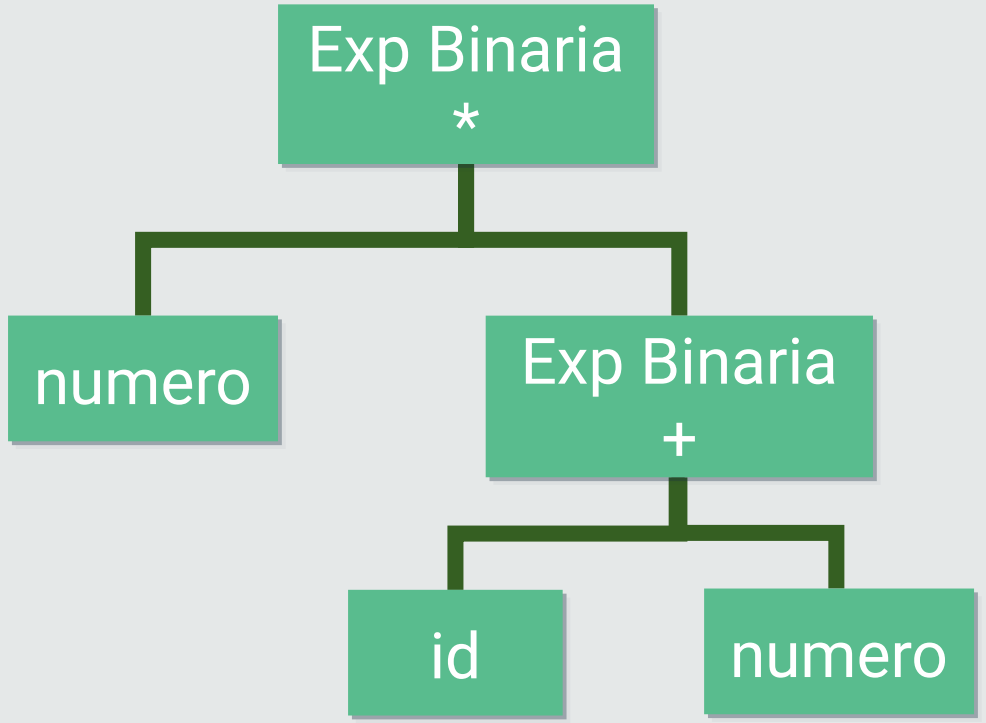
Arboles Sintácticos Abstractos

- Diferencias entre el árbol de derivación y el AST

E → E+E | E*E |
(E) | id | num

```
...
4*(b+2)
...
```





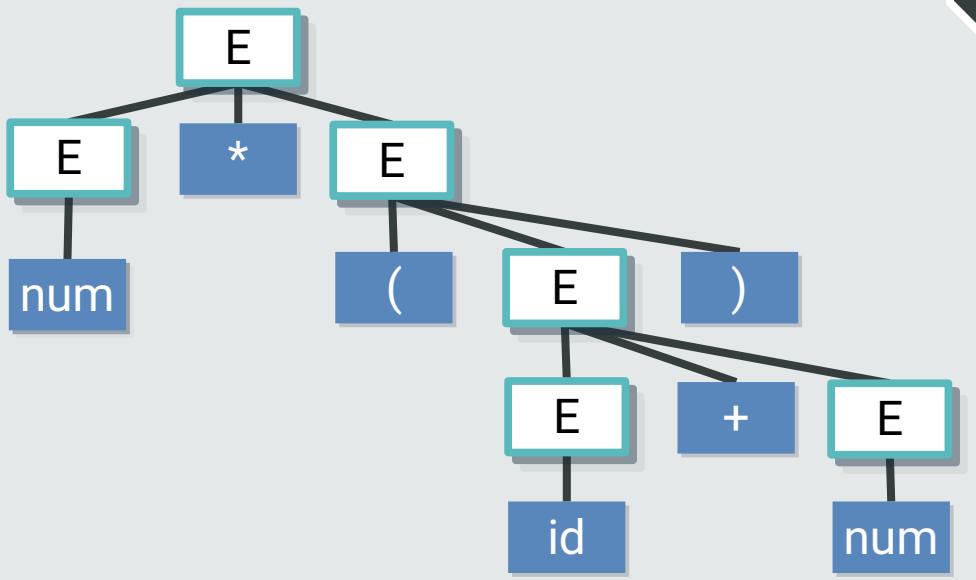
Arboles Sintácticos Abstractos

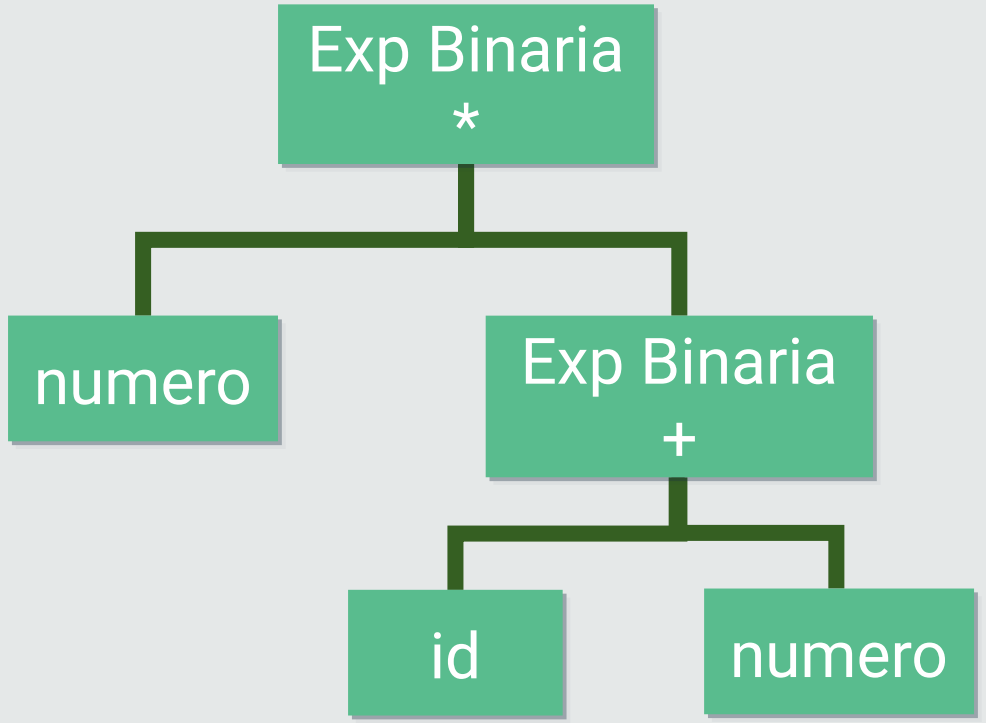
Esta gramática es ambigua… vemos como sería el AST con una no ambigua...

- Diferencias entre [illegible] y el AST

E → E+E | E*E | (E) | id | num

```
…
4*(b+2)
…
```





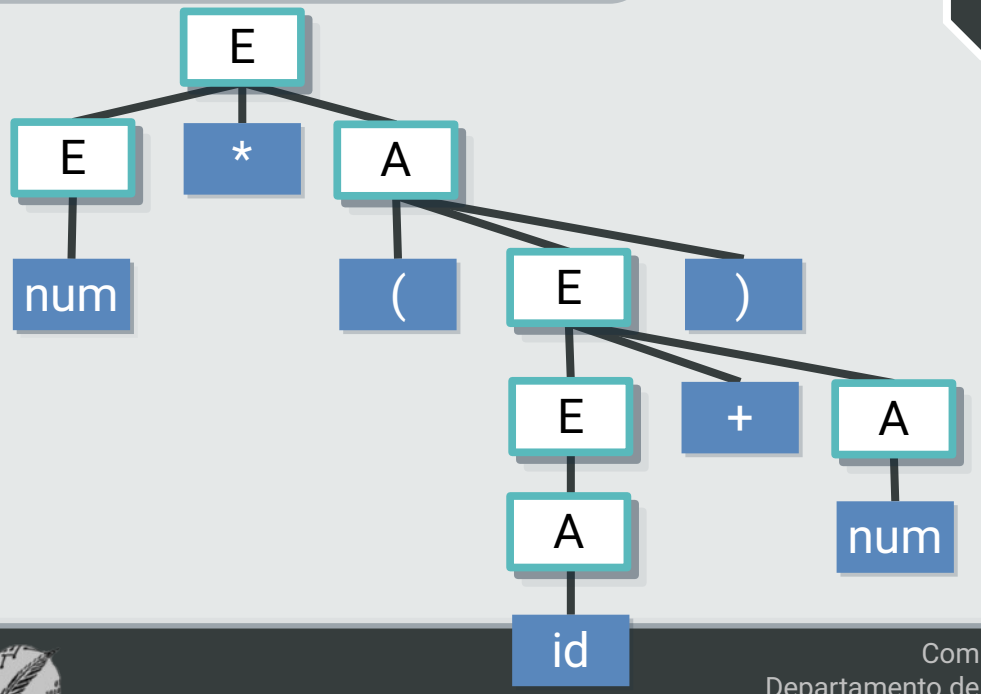
Arboles Sintácticos Abstractos

- Diferencias ent ón y el AST

Esta gramática tiene recursión a Izquierda… veamos como sería el AST con una gramática equivalente sin ese problema

E → E+A | E*A | A
A → (E) | id | num

```
…
4*(b+2)
…
```



Arboles Sintácticos Abstractos

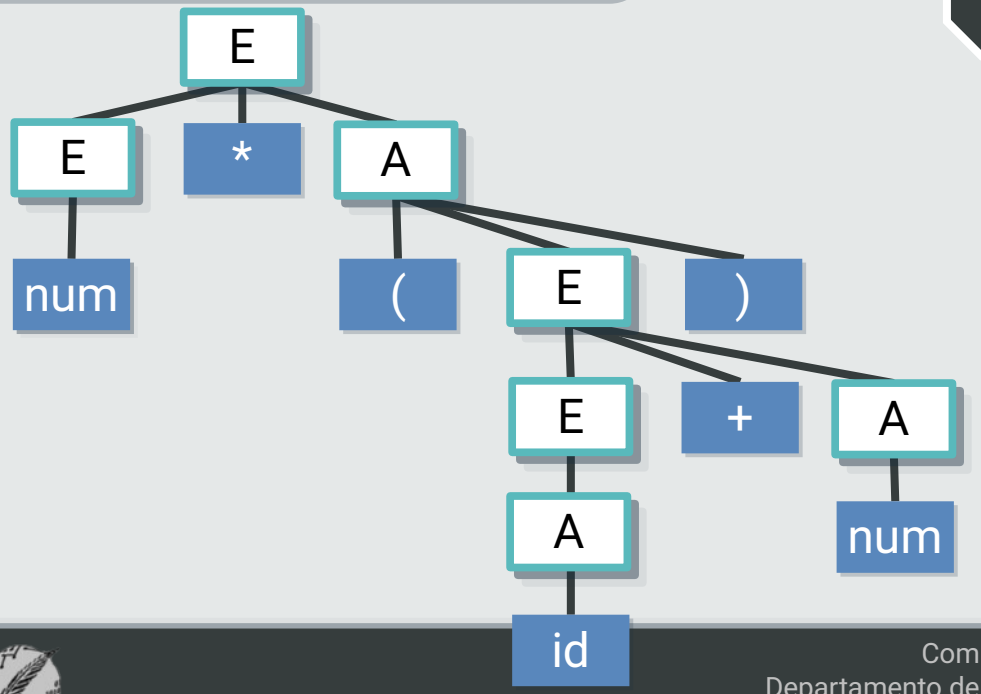
- Diferencias ent ón y el AST

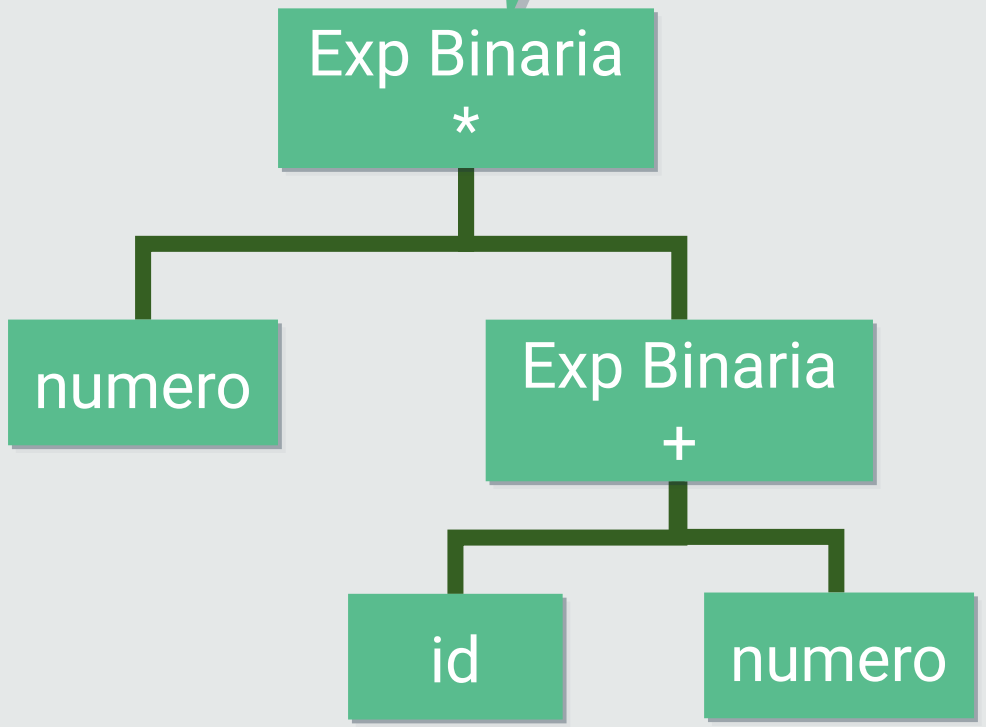
Esta gramática tiene recursión a Izquierda… veamos como sería el AST con una gramática equivalente sin ese problema

Es Igual!

E → E+A | E*A | A
A → (E) | id | num

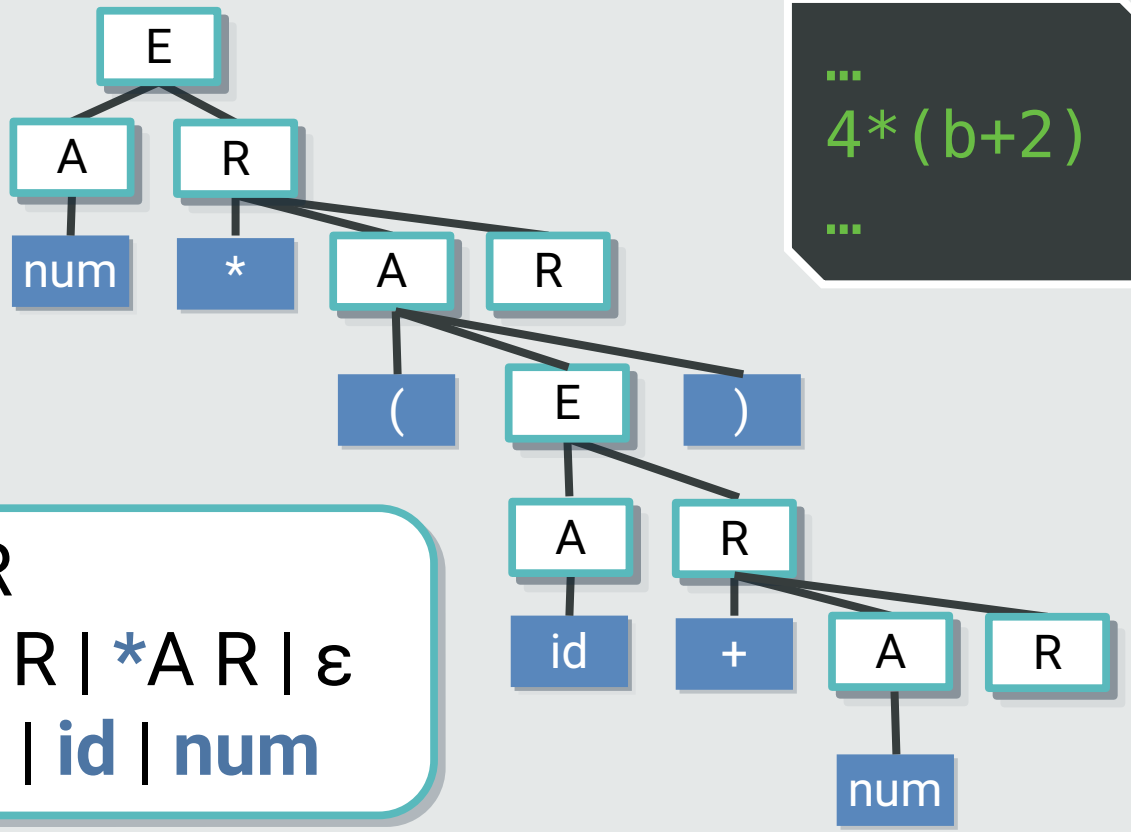
```
…
4*(b+2)
…
```





Arboles Sintácticos Abstractos

- Diferencias entre el árbol de derivación y el AST

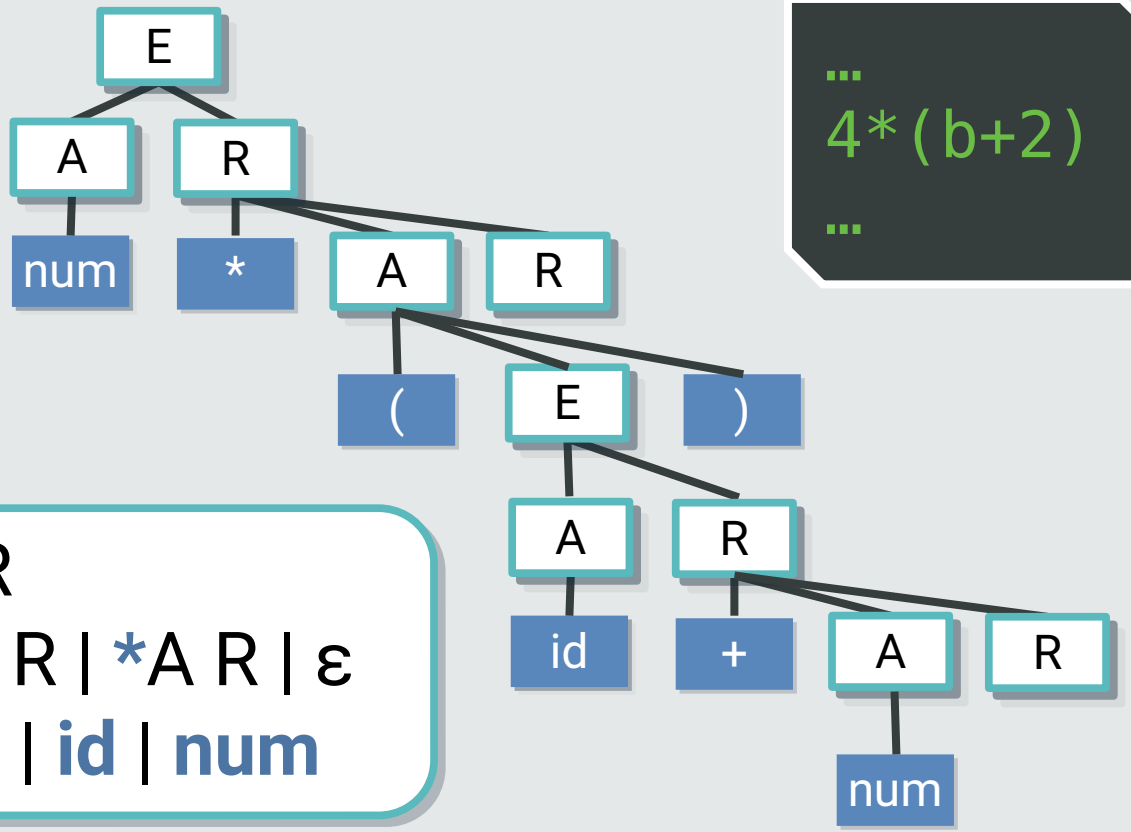



```
...
4*(b+2)
...
```

E → A R
R → +A R | *A R | ε
A → (E) | id | num

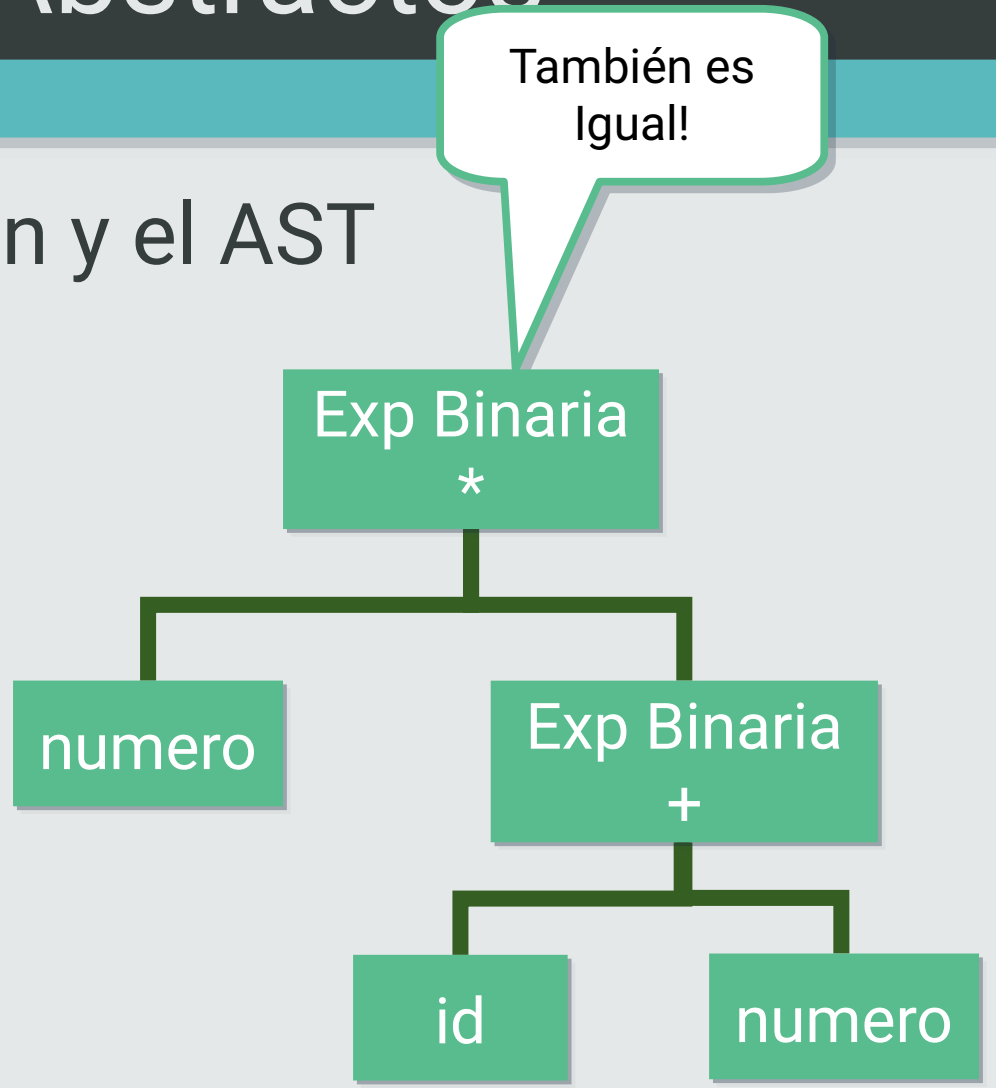
Arboles Sintácticos Abstractos

- Diferencias entre el árbol de derivación y el AST




```
...
4*(b+2)
...
```

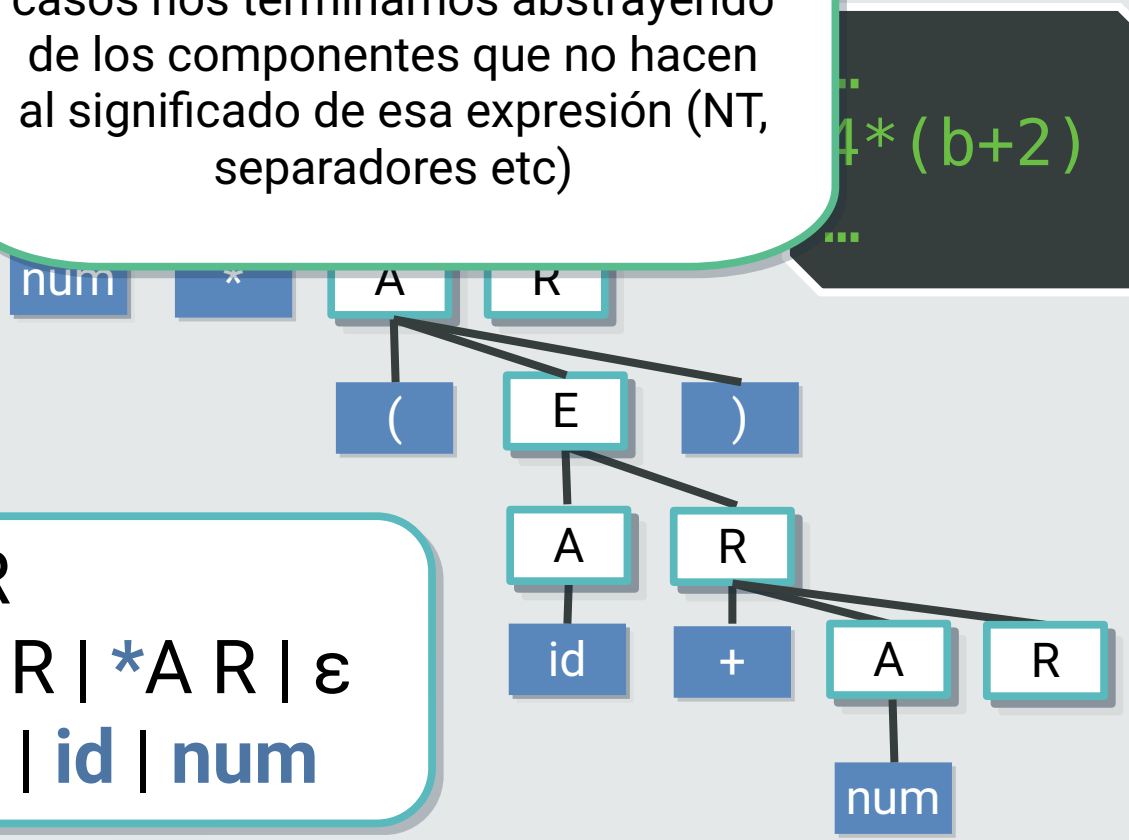
E → A R
R → +A R | *A R | ε
A → (E) | id | num



Arboles Sintácticos Abstractos

- Dife e derivación y el AST

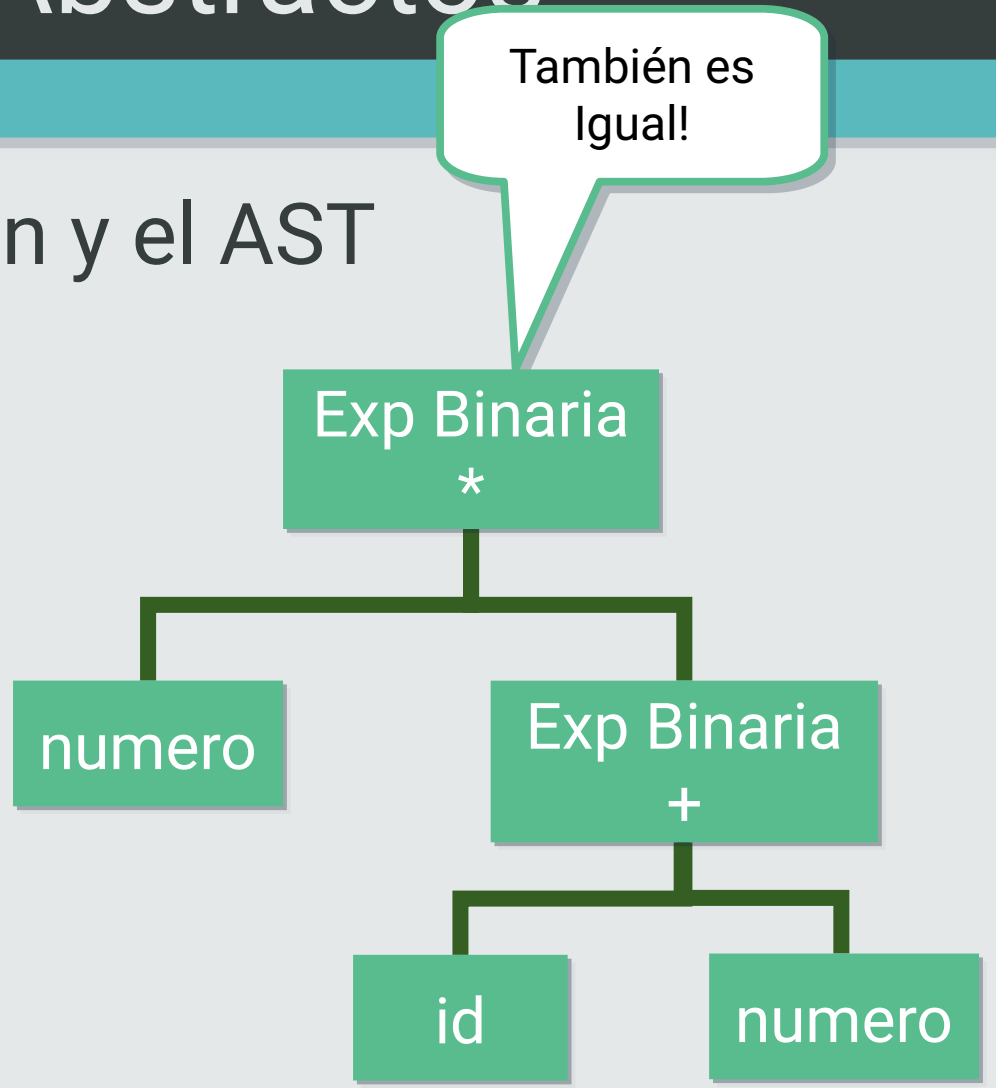
Como podemos ver los tres ASTs son iguales por que en los tres casos nos terminamos abstrayendo de los componentes que no hacen al significado de esa expresión (NT, separadores etc)



$E \rightarrow A\ R$

$R \rightarrow +A\ R \mid *A\ R \mid \varepsilon$

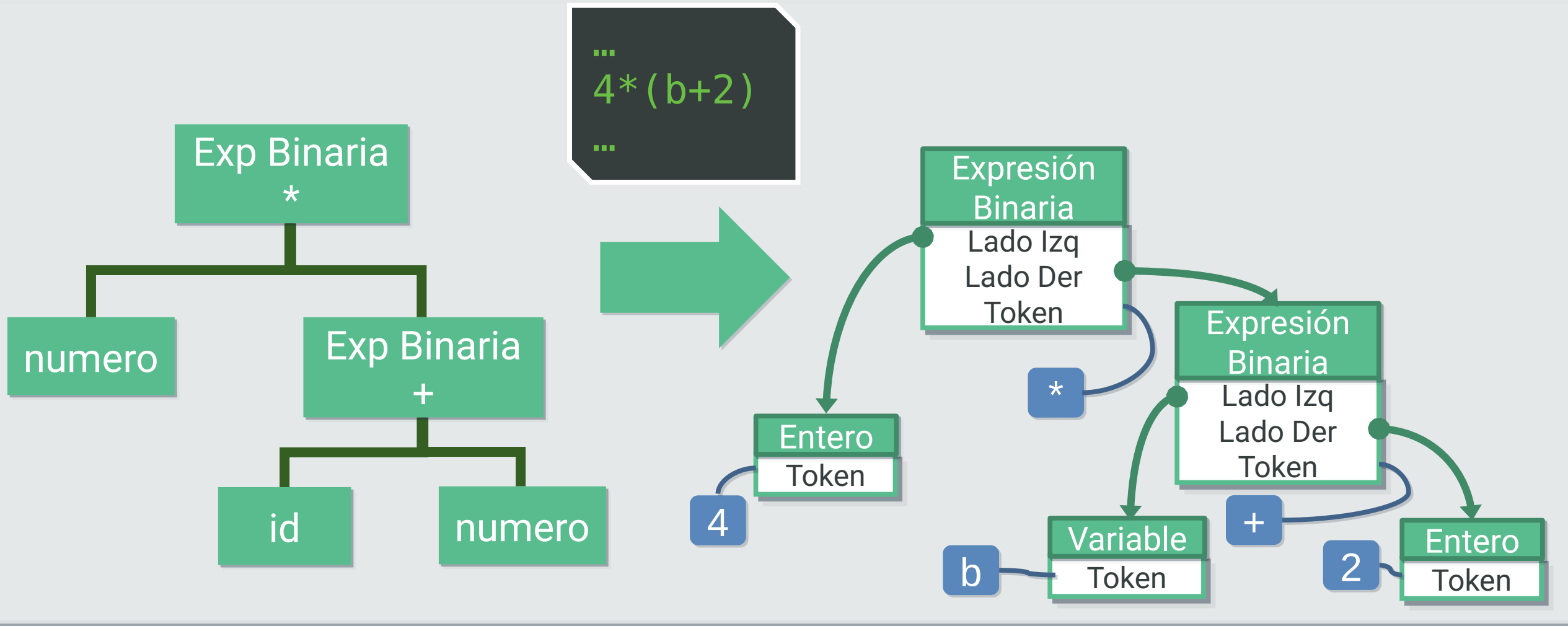
$A \rightarrow (E) \mid id \mid num$



Arboles Sintácticos Abstractos

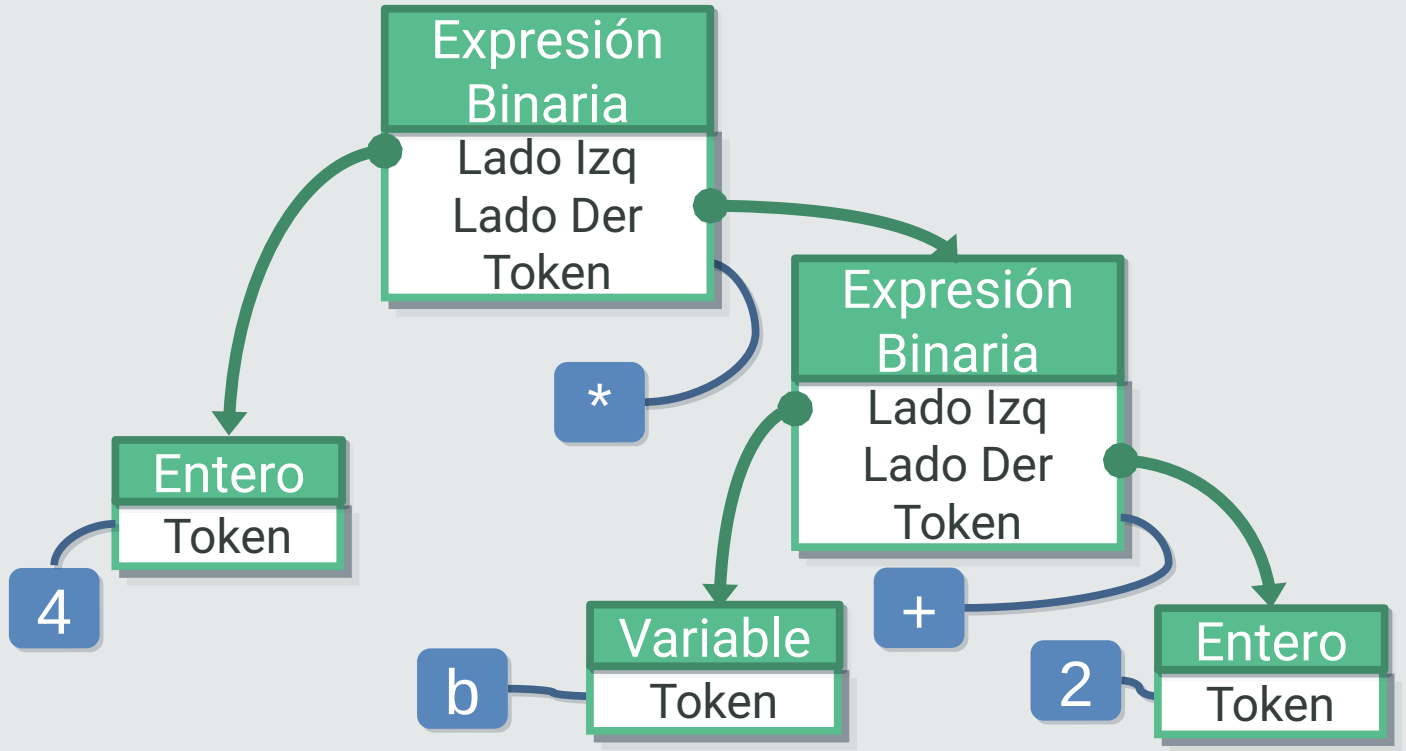
- Una de las **características** mas importantes de los ASTs es que podemos **enriquecerlos** con toda la **información** que creamos **adecuada** para el **análisis semántico**
- Por ejemplo:
  - En los **nodos literal** podemos asociar el **token** que caracteriza al literal
  - En los **nodos variable** podemos asociar el **token** que vinculado al id
  - **Distinguir** de manera **diferente** a los **hijos** de un nodo

Arboles Sintácticos Abstractos



Arboles Sintácticos Abstractos

```
…
a = 4*(b+2);
…
```



Arboles Sintácticos Abstractos

```
…
a = 4*(b+2);
if (a>6) return a;
…
```

Arboles Sintácticos Abstractos

```
{
a = 4*(b+2);
if (a>6) return a;
}
```

Arboles Sintácticos Abstractos

- ¿Qué pasa cuando tenemos variables, llamadas, parámetros y encadenados como en MiniJava?

```
v.m1(9,4*a).w+5
```



Árbol Sintáctico Abstracto (AST) – Diseño

Diseño del AST

- La clave para **diseñar e implementar un AST** es identificar que **tipos de nodos** va a tener el árbol
- Idealmente, cada **tipo de constructor sintáctico** que tiene "**significado**" dentro del programa es **caracterizado** por un **tipo de nodo**
- Por ej: bloque, asignación, expresión binaria, while, return, etc.
- A cada **tipo de nodo** lo van a **caracterizar**:
  - Su "**significado**" como entidad en un programa
  - los **hijos** que puede tener (la cantidad y el tipo de nodos que pueden ser)
  - **Información** vinculada al nodo que utilizaremos para el **análisis semántico**

Diseño del AST

- Intuitivamente, a **cada tipo de nodo** lo podemos ver como una **clase** de un **lenguaje orientado a objetos**

- Las **variables de instancia** de la clase **indican** cuáles son sus **hijos en el árbol** y otros atributos del nodo



Diseño del AST



Diseño del AST



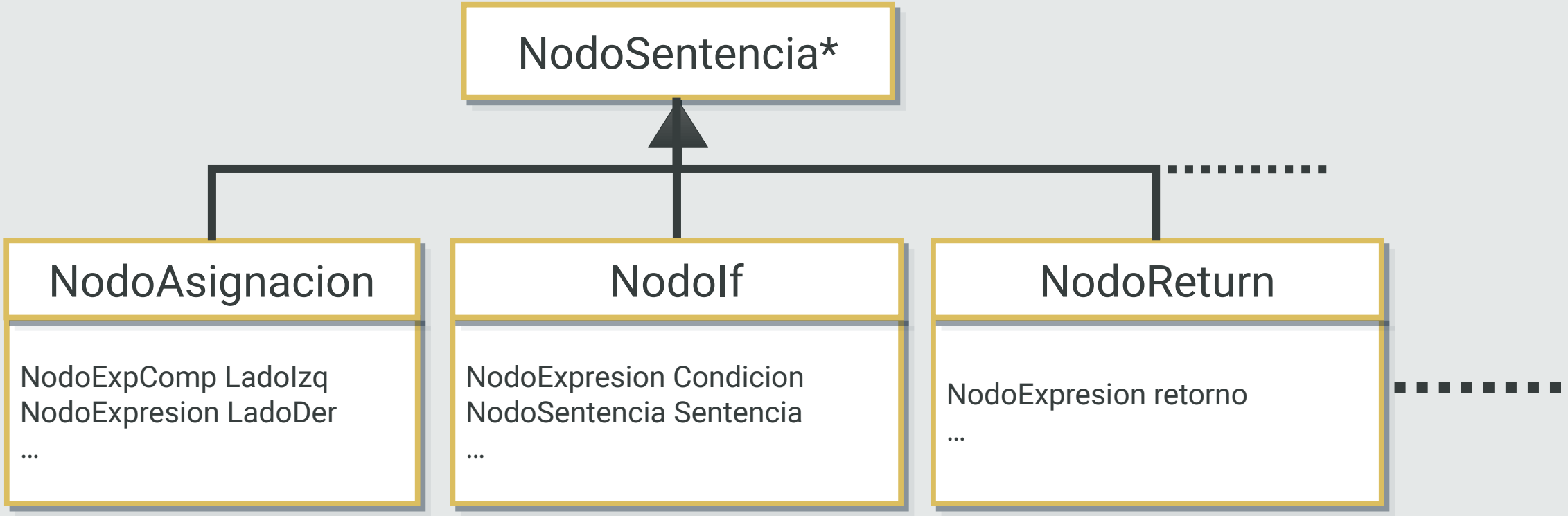
Un nodo **Expresión** puede ser un **literal**, un **ID**, una **llamada** o una **expresión compleja** con varios operadores

Puede ser **cualquier tipo de sentencia**: Asignación, Bloque, While, etc.

Nos podemos valer de herencia para modelar estas relaciones.

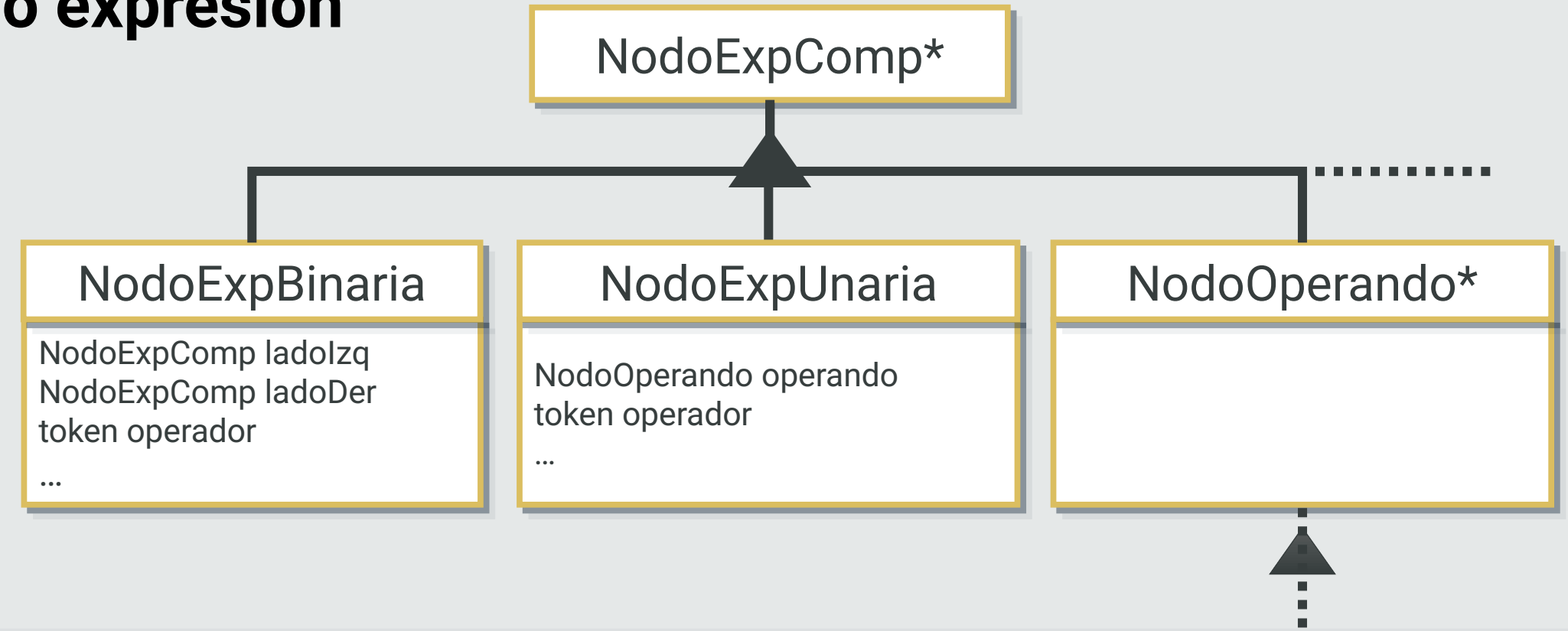
Diseño del AST

- Por ejemplo, para que un **nodo sentencia** pueda estar **instanciado** con **cualquier tipo de sentencia**:



Diseño del AST

- De manera similar, podemos modelar los diferentes tipos de **nodo expresión**

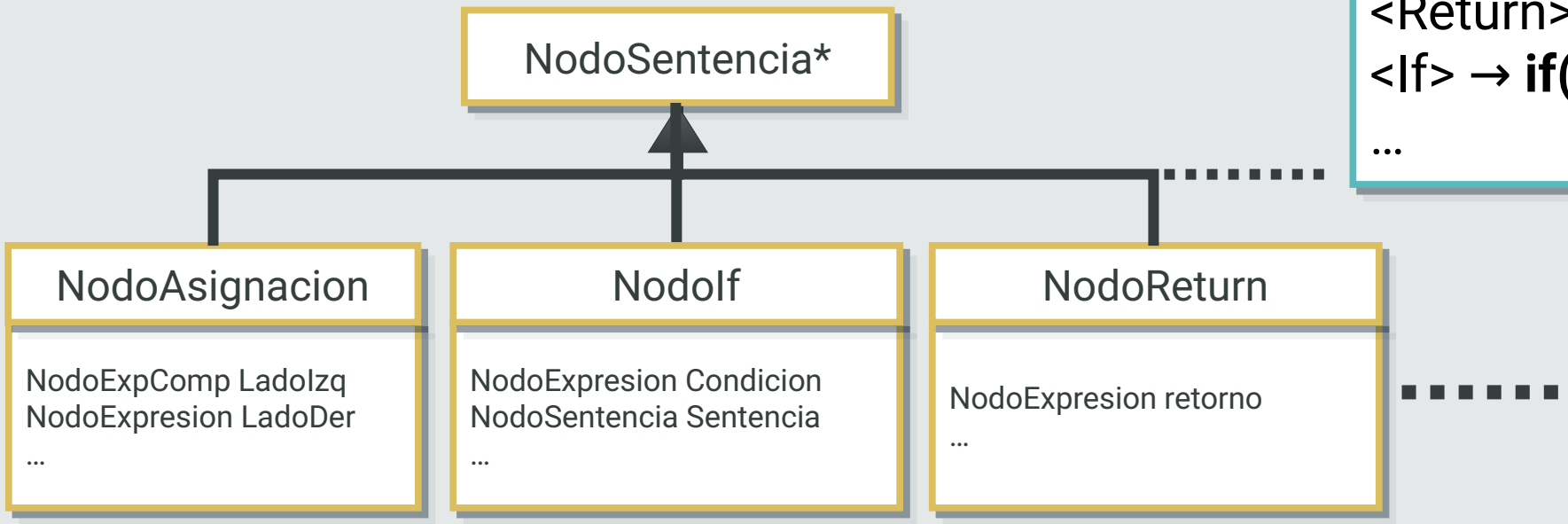


Diseño del AST

- En general…

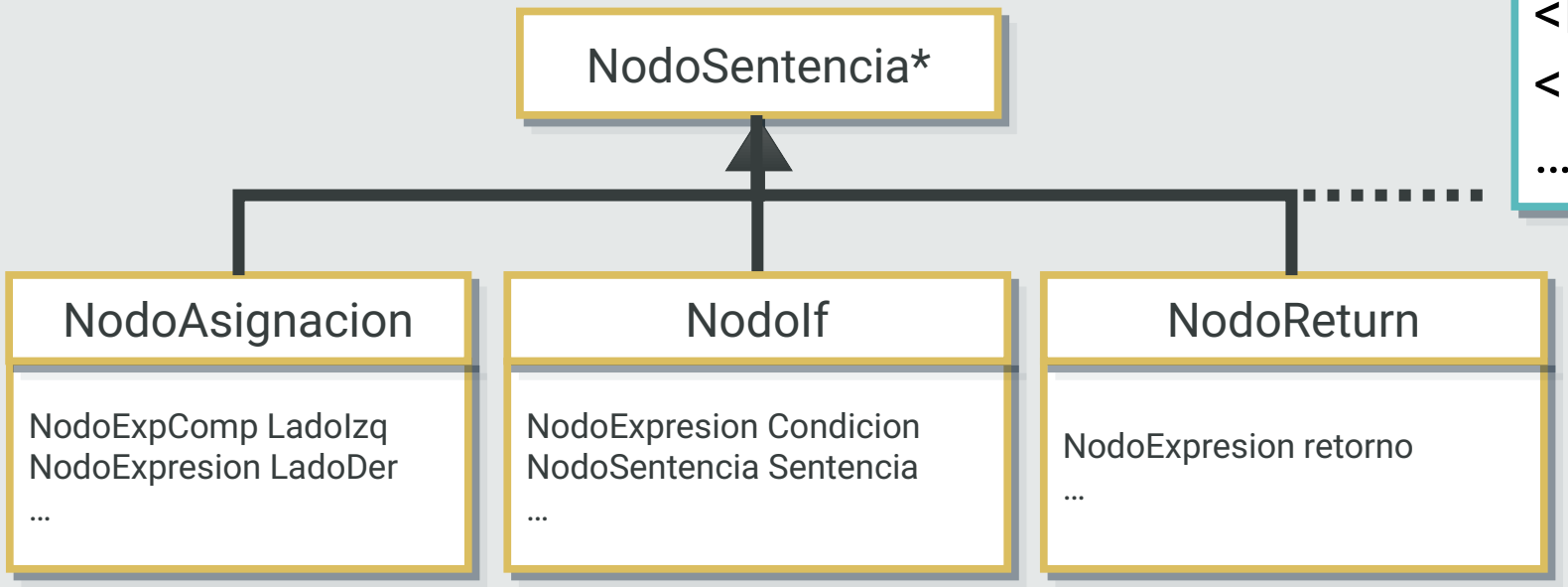
<Sentencia> → <Asignación>;
<Sentencia> → <Return>;
<Sentencia> → <If>

<Return> → **return** <ExpresionOVacio>
<If> → **if**(<Expresion>) <Sentencia>
…



Diseño del AST

- En general…



<Sentencia> → <Asignación>;
<Sentencia> → <Return>;
<Sentencia> → <If>

<Return> → **return** <ExpresionOVacio>
<If> → **if**(<Expresion>) <Sentencia>
…

Gramática **original MiniJava**

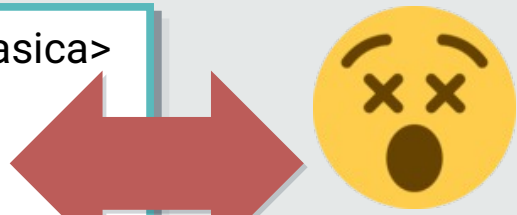
Nos da una guía de como diseñar el AST Para **MiniJava**

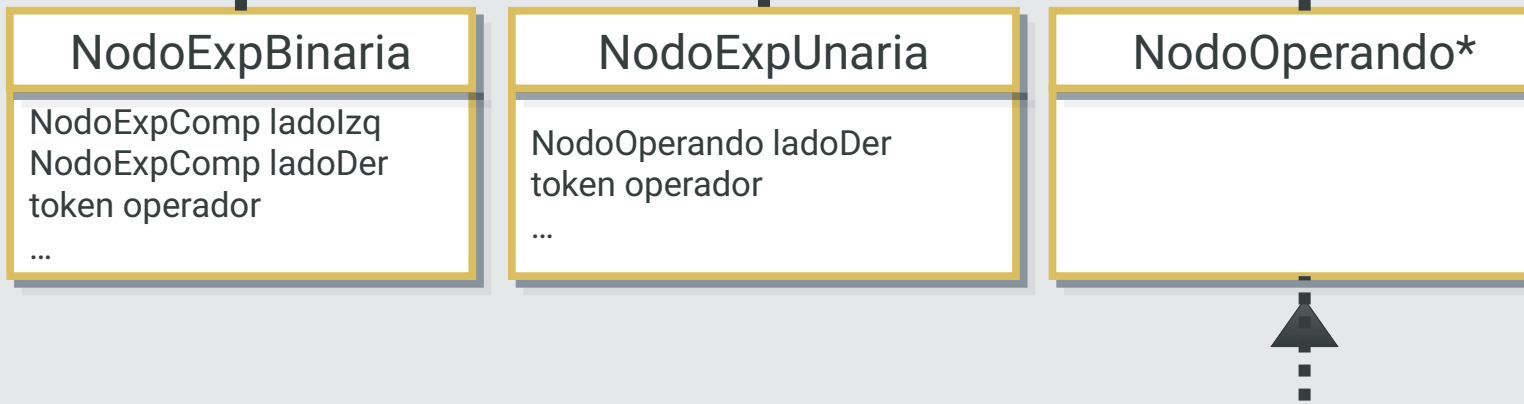
AST – Implementación

- Aun que no siempre…

```
<ExpresionComp>→<ExpresionComp> <OperadorBinario> <ExpresionBasica>
<ExpresionComp>→<ExpresionBasica>
<ExpresionBasica>→ <OperadorUnario><Operando>
<ExpresionBasica>→ <Operando>
…
```

Gramática **original MiniJava**





Diseño del AST

- El **problema** viene de que **en la gramática original** de MiniJava **ya eliminamos la ambigüedad** en las expresiones
  - Codificando sintácticamente la **precedencia** expresiones binarias, unarias y operandos

<ExpresionComp>→<ExpresionComp><OperadorBinario><ExpresionComp>
<ExpresionComp>→<OperadorUnario><Operando>
<ExpresionComp> →<Operando>
…

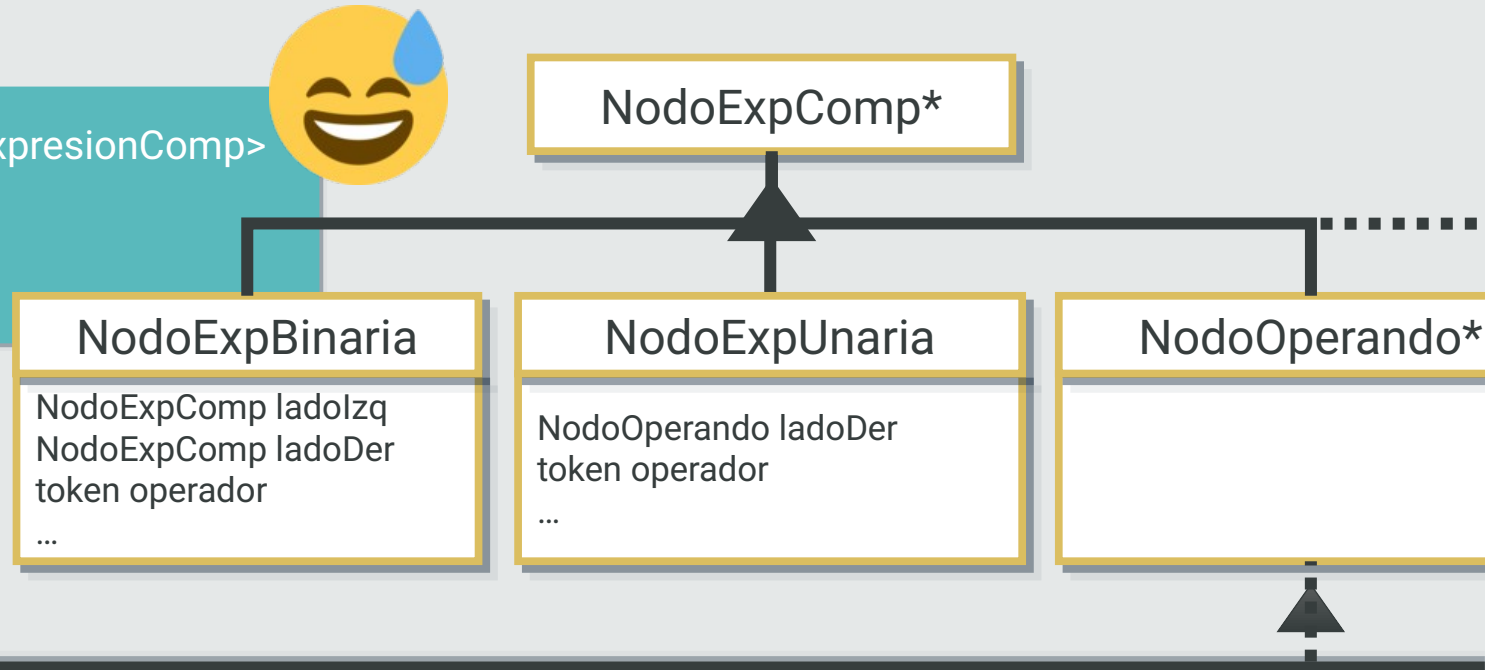


<ExpresionComp>→<ExpresionComp> <OperadorBinario> <ExpresionBasica>
<ExpresionComp>→<ExpresionBasica>
<ExpresionBasica>→ <OperadorUnario><Operando>
<ExpresionBasica>→ <Operando>
…

Diseño del AST

- El **problema** viene de que **en la gramática original** de MiniJava **ya eliminamos la ambigüedad** en las expresiones
  - Codificando sintácticamente la **precedencia** expresiones binarias, unarias y operandos

<ExpresionComp>→<ExpresionComp><OperadorBinario><ExpresionComp>
<ExpresionComp>→<OperadorUnario><Operando>
<ExpresionComp> →<Operando>
…



Diseño del AST

- Entonces, una **buena heurisitica** para diseñar el AST es **mirar la gramática original** del lenguaje y diseñar los distintos **tipos de nodo** base a sus **reglas**
- Aun así es importante entender que **representa** del programa cada **nodo**
  - este **entendimiento** puede hacernos **modelar** algún nodo de una forma un poco **diferente** (como en las expresiones) al de la **gramática**

- El objetivo final es tener una representación adecuada de cada tipo de sentencia y sus componentes internos

Creación del AST

AST– Creando los ASTs

- Los **AST** se **crearán** a medida que se va **realizando** el **Análisis Sintáctico**.
- Haremos una **EDT** sobre la gramática utilizada en el Analizador Sintáctico donde las **acciones semánticas** construirán los **ASTs**
- Luego, modificaremos los **métodos** del Analizador Sintáctico para reflejar las **acciones semánticas** de la EDT

AST– Creando los ASTs

- Por ejemplo, consideremos la regla:

<Sentencia> …|**if(**<Expresion>**)** Sentencia|…

EDT

<Sentencia>$_1$

… |

if (<Expresion>**)** <Sentencia>$_2$

AST– Creando los ASTs

- Por ejemplo, consideremos la regla:

<Sentencia> …|**if(**<Expresion>**)** Sentencia|…

EDT

<Sentencia>$_1$
… |
if (<Expresion>**)** <Sentencia>$_2$
{
<Sentencia>$_1$**.nodo** =
new NodoIfSolo(*<Expresion>*.**nodo**,
<Sentencia>$_2$**.nodo**);
}
|…

<Sentencia> y **<Expresion>** tienen el atributo **nodo** para devolver el AST del código que reconocen

AST– Creando los ASTs

- Por ejemplo, consideremos la regla:

<Sentencia> …|**if(**<Expresion>**)** Sentencia|…

EDT

<Sentencia>$_1$
… |
if (<Expresion>**)** <Sentencia>$_2$
{
<Sentencia>$_1$**.nodo** =
new NodoIfSolo(*<Expresion>*.**nodo**,
<Sentencia>$_2$**.nodo**);
}
|…

<Sentencia> y **<Expresion>** tienen el atributo **nodo** para devolver el AST del código que reconocen